Vue 프로젝트 기본 정리

프로젝트 생성 및 실행

```bash
npm create vue@latest
# 프로젝트 이름: vue-examples
cd vue-examples
npm install
npm run dev

npm init vue
npm create vue@latest vue-examples
```

> 공식 문서

---

1. Vue 프로젝트 생성

Vue 공식 프로젝트 생성 도구(`create-vue`)를 사용한다.

```bash
npm create vue@latest
cd 프로젝트명
```

실행하면 아래와 같은 옵션을 선택할 수 있다.

옵션	설명
TypeScript	타입 기반 JavaScript 사용
Router	SPA 라우팅 기능
Pinia	상태 관리 라이브러리
Vitest	단위 테스트 도구
ESLint	코드 오류 방지
Prettier	코드 스타일 자동 정리

---

2. 패키지 설치

```bash
npm install
```

`package.json`에 정의된 라이브러리를 설치한다.
설치 결과 `node_modules` 폴더가 생성된다.

---

3. 개발 서버 실행

```bash
npm run dev
```

Vue 개발 서버를 실행한다.

- 실행 후 접속 주소: `http://localhost:5173`

---

4. package.json

프로젝트 설정 파일. `scripts`에 실행 명령이 정의되어 있다.

```json
{
  "name": "vue-examples",
  "version": "0.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "vue": "^3.5.29"
  },
  "devDependencies": {
    "@vitejs/plugin-vue": "^6.0.4",
    "vite": "^7.3.1"
  }
}
```

명령어	실행 방법	설명
`dev`	`npm run dev`	개발 서버 실행
`build`	`npm run build`	배포용 빌드 생성
`preview`	`npm run preview`	빌드 결과 확인

dependencies vs devDependencies

구분	필드	설명
실행 시 필요	`dependencies`	`vue` 등
개발 시만 필요	`devDependencies`	`vite`, `@vitejs/plugin-vue` 등

---

5. Vue 프로젝트 기본 구조

```
project
├── node_modules
├── public
├── src
|   ├── assets
|   ├── components
|   ├── App.vue
|   └── main.js
├── .gitignore
├── index.html
├── jsconfig.json
├── package-lock.json
├── package.json
├── README.md
└── vite.config.js
```

경로	설명
`node_modules`	npm install로 설치된 모든 라이브러리
`public`	빌드 시 그대로 복사되는 정적 파일 (이미지, favicon 등)
`src`	Vue 애플리케이션의 실제 코드가 작성되는 핵심 폴더
`src/assets`	이미지, CSS, 폰트 등 리소스 파일
`src/components`	재사용 가능한 Vue 컴포넌트
`src/App.vue`	루트(최상위) 컴포넌트
`src/main.js`	Vue 앱을 생성하고 시작하는 파일
`.gitignore`	Git 버전 관리 제외 파일 목록
`index.html`	Vue 앱이 렌더링되는 HTML 템플릿
`jsconfig.json`	VSCode 경로 자동완성 및 JS 프로젝트 설정
`package-lock.json`	설치된 패키지의 정확한 버전을 고정하는 파일. Git에 포함 권장
`package.json`	프로젝트 정보, 패키지, 스크립트 정의
`README.md`	프로젝트 설명 및 사용 방법 문서
`vite.config.js`	Vite 개발 서버 및 빌드 설정

---

6. src 폴더

Vue 코드가 작성되는 핵심 폴더.

```
src
├── assets        ← 이미지, CSS 등 정적 리소스
├── components    ← 재사용 가능한 컴포넌트
├── App.vue       ← 루트 컴포넌트
└── main.js       ← 앱 시작 파일
```

---

7. main.js

Vue 앱의 시작 파일 (entry point)

- `createApp()` → Vue 애플리케이션 생성
- `mount("#app")` → `index.html`의 `<div id="app"></div>`에 연결

```html
<!-- index.html -->
<div id="app"></div>
```

```js
// main.js
import { createApp } from 'vue'
import App from './App.vue'

createApp(App).mount('#app')
```

---

8. App.vue

Vue 프로젝트의 루트 컴포넌트 (모든 컴포넌트의 최상위 부모).
모든 컴포넌트는 `App.vue`에서 시작된다.

---

9. components 폴더

재사용 가능한 Vue 컴포넌트를 저장하는 폴더.

```
components
├── HelloWorld.vue
├── TodoList.vue
└── UserCard.vue
```

---

10. Single File Component (SFC)

Vue는 하나의 `.vue` 파일에서 HTML, JavaScript, CSS를 함께 작성할 수 있다.

구성요소	역할
`<template>`	화면 구조
`<script setup>`	로직 (Vue 3 권장 방식)
`<style>`	CSS

Options API (Vue 2 방식)

```vue
<template>
  <h1>{{ title }}</h1>
</template>

<script>
export default {
  data() {
    return {
      title: 'Hello Vue!'
    }
  }
}
</script>

<style>
h1 { color: blue; }
</style>
```

Composition API + script setup (Vue 3 권장 방식)

```vue
<template>
  <h1>{{ title }}</h1>
</template>

<script setup>
import { ref } from 'vue'

const title = ref('Hello Vue!')
</script>

<style>
h1 { color: blue; }
</style>
```

> Vue 3에서는 `<script setup>` 방식을 공식 권장한다.

---

11. Vue 앱 실행 흐름

```
package.json
      ↓
npm install         → node_modules 생성
      ↓
npm run dev         → vite 명령 실행
      ↓
Vite 개발 서버 실행
      ↓
브라우저 접속 (http://localhost:5173)
      ↓
index.html
      ↓
main.js             → createApp(App).mount('#app')
      ↓
App.vue
      ↓
components
```

[참고] Vite란?

> 빌드 = 개발 중인 Vue 프로젝트를 실제 배포 가능한 형태로 만드는 작업

Vite는 Vue 프로젝트의 개발 서버이자 빌드(번들링) 도구이다.

기능	설명
개발 서버 실행	로컬에서 앱 확인 가능
자동 새로고침	코드 변경 시 즉시 반영
빠른 빌드	ES Module 기반으로 빠른 처리

[참고] package-lock.json이란?

설치된 패키지의 **정확한 버전을 고정**하는 파일.

- `package.json`의 `"vue": "^3.5.29"`는 3.5.29 이상이면 설치 가능하다는 의미
- `package-lock.json`은 실제로 설치된 버전(예: 3.5.29)을 정확히 기록한다
- 팀원 모두가 `npm install` 시 **동일한 버전**을 설치할 수 있다

> `package-lock.json`은 Git에 포함하는 것을 권장한다.

[참고] 번들링이란?

번들링(Bundling)이란 여러 개의 JS, CSS, 이미지 파일을 하나의 묶음으로 만드는 과정이다.
브라우저에서 효율적으로 로딩하기 위해 사용하며, Vite가 이 역할을 담당한다.

도구	설명
Vite	Vue 3 공식 권장 번들러
Webpack	구버전 Vue 프로젝트에서 사용